

Inner Product Spaces and Homography

A.M. Krus

October 16, 2025

1 Introduction

This project explores how to determine the pointing direction of a gun-like controller equipped with an infrared (IR) camera. The objective is to map points detected in the camera frame to their corresponding positions on a display. While the computational solution uses a homography, the mathematical context is best understood through the framework of inner product spaces, which provide notions of angle, length, distance, and orthogonal projection. Section 1, 2 and 3 provide the mathematical context from linear algebra and numerical analysis. Later section explore the concrete application of homography. References are provided for full conceptual explanations.

2 Inner Product Spaces

Let $V = \mathbb{R}^n$ be a real vector space. An *inner product* is a function $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{R}$ satisfying linearity, symmetry, and positivity. For $V = \mathbb{R}^3$ with the standard Euclidean inner product,

$$\langle \mathbf{x}, \mathbf{y} \rangle = x_1y_1 + x_2y_2 + x_3y_3.$$

From the inner product we derive:

- the induced norm: $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$,
- the distance: $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$,
- the angle: $\cos \theta = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \|\mathbf{y}\|}$.

These notions allow us to interpret pixels as rays, compare pointing directions via angles, and measure reprojection errors via distances.

3 Orthogonal Projection

A fundamental operation in an inner product space is the *orthogonal projection*. Given a vector $\mathbf{x} \in \mathbb{R}^3$ and a unit vector $\mathbf{u}$, the projection of $\mathbf{x}$ onto the line spanned by $\mathbf{u}$ is

$$\text{proj}_{\mathbf{u}}(\mathbf{x}) = \frac{\langle \mathbf{x}, \mathbf{u} \rangle}{\langle \mathbf{u}, \mathbf{u} \rangle} \mathbf{u}.$$

For projection onto a plane with orthonormal basis $\{\mathbf{u}, \mathbf{v}\}$,

$$\text{proj}_{\text{span}\{\mathbf{u}, \mathbf{v}\}}(\mathbf{x}) = UU^T \mathbf{x}, \quad U = [\mathbf{u} \ \mathbf{v}].$$

In the context of this project:

- each pixel (u, v) corresponds to a ray $\mathbf{r}(u, v) \in \mathbb{R}^3$,
- the screen is modeled as a 2D plane in $\mathbb{R}^3$,
- finding where a ray intersects the screen can be viewed as projecting the ray onto the plane of the screen.

Thus, orthogonal projection is the geometric mechanism connecting camera rays to screen coordinates. The homography, introduced next, encodes this relationship algebraically.

4 Camera Rays in $\mathbb{R}^3$

In the pinhole camera model, each pixel (u, v) corresponds to a normalized ray:

$$\mathbf{r}(u, v) = \frac{1}{\sqrt{x^2 + y^2 + 1}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad x = \frac{u - c_x}{f_x}, \quad y = \frac{v - c_y}{f_y}.$$

5 Homography as a Linear Operator

The display surface is planar and can be parameterized by coordinates (X, Y) . Given four calibration points on the screen and their detected counterparts (u, v) in the camera, there exists a projective mapping (homography):

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = H \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}.$$

While orthogonal projection describes the geometry, the homography provides the computational mechanism to carry out the mapping.

6 Homography Estimation

Each correspondence $(X, Y) \leftrightarrow (u, v)$ yields two linear equations:

$$\begin{aligned} Xh_{11} + Yh_{12} + h_{13} - u(Xh_{31} + Yh_{32} + h_{33}) &= 0, \\ Xh_{21} + Yh_{22} + h_{23} - v(Xh_{31} + Yh_{32} + h_{33}) &= 0. \end{aligned}$$

Stacking four correspondences yields an 8×8 linear system

$$A\mathbf{h} = \mathbf{b},$$

with unknown vector

$$\mathbf{h} = [h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}]^T.$$

7 Solution via Gaussian Elimination

The linear system can be solved using Gaussian elimination, reducing the augmented matrix $[A|\mathbf{b}]$ to row echelon form. This yields the homography matrix H . The quality of the solution can be assessed using inner product tools, e.g., the reprojection error

$$E = \sum_i \|(u_i, v_i) - H(X_i, Y_i, 1)^T\|.$$

8 Homography Variants by Point Constraints

The computation of a planar mapping depends on the number of available correspondences between world points (X_i, Y_i) and camera detections (u_i, v_i) . Since a homography has eight degrees of freedom, four correspondences are sufficient to fully determine it. Fewer correspondences constrain simpler models that approximate the homography in a lower-dimensional subspace.

8.1 Four-Point Case: Full Projective Homography

Given four non-collinear points, we obtain eight independent equations in eight unknowns. The system

$$A\mathbf{h} = \mathbf{b}$$

is square and can be solved exactly via Gaussian elimination:

$$\mathbf{h} = A^{-1}\mathbf{b}.$$

This yields the full projective homography H , which encodes translation, rotation, scale, shear, and perspective distortion.

Listing 1: 4-point homography estimation

```
function computeHomography4(src [4][2], dst [4][2]):
    build 8x8 system A, b from correspondences
    if solve8x8(A, b) succeeds:
        fill H matrix from solution
        return H
    else:
        return error
```

8.2 Three-Point Case: Affine Transformation

With three points, the system becomes underdetermined for a full projective mapping. By constraining $h_{31} = h_{32} = 0$ and $h_{33} = 1$, we restrict H to the affine form:

$$H = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}.$$

Each correspondence contributes two equations, giving six equations for six unknowns. The affine model preserves parallelism and is sufficient when perspective effects are negligible (e.g., camera nearly perpendicular to the screen).

Mathematical formulation.

$$\begin{aligned} u_i &= a_{11}X_i + a_{12}Y_i + t_x, \\ v_i &= a_{21}X_i + a_{22}Y_i + t_y. \end{aligned}$$

Stacking the three pairs yields a 6×6 linear system, solved directly by Gaussian elimination.

Listing 2: 3-point affine transformation estimation

```
function computeAffine3(src [3][2], dst [3][2]):
    build 6x6 system A, b assuming h31 = h32 = 0, h33 = 1
    if solve6x6(A, b) succeeds:
        construct H = [[a11, a12, tx], [a21, a22, ty], [0, 0, 1]]
        return H
    else:
        return error
```

8.3 Two-Point Case: Similarity Transform

With only two correspondences, we cannot determine a unique homography. Instead, we assume an isotropic similarity model combining uniform scale s , rotation $R(\theta)$, and translation $\mathbf{t}$:

$$H = \begin{bmatrix} s \cos \theta & -s \sin \theta & t_x \\ s \sin \theta & s \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix}.$$

This model preserves distances up to scale and assumes the scene lies approximately fronto-parallel to the camera.

Mathematical formulation. Let $\mathbf{p}_1, \mathbf{p}_2$ be world points and $\mathbf{q}_1, \mathbf{q}_2$ their corresponding image points. The scale and rotation are derived as

$$s = \frac{\|\mathbf{q}_2 - \mathbf{q}_1\|}{\|\mathbf{p}_2 - \mathbf{p}_1\|}, \quad R = R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix},$$

where θ is the angle between the point pairs. The translation follows as

$$\mathbf{t} = \mathbf{q}_1 - sR\mathbf{p}_1.$$

Listing 3: 2-point similarity transform estimation

```
function computeSimilarity2(src [2][2], dst [2][2]):
    dp = src [1] - src [0]
    dq = dst [1] - dst [0]
    s = norm(dq) / norm(dp)
    theta = atan2(dq.y, dq.x) - atan2(dp.y, dp.x)
    R = [[cos(theta), -sin(theta)], [sin(theta), cos(theta)]]
    t = dst [0] - s * R * src [0]
    construct H = [[sR, t], [0, 0, 1]]
    return H
```

8.4 Remarks on Least Squares Estimation

When more than four correspondences are available, or when measurements contain noise, the equations $A\mathbf{h} = \mathbf{b}$ become inconsistent. In such cases, the best-fit homography minimizes the reprojection error in the least-squares sense:

$$\min_{\mathbf{h}} \|A\mathbf{h} - \mathbf{b}\|^2,$$

with normal equations

$$A^T A \mathbf{h} = A^T \mathbf{b}.$$

This approach produces a statistically optimal estimate under Gaussian noise assumptions and generalizes the Gaussian elimination solver to overdetermined systems.

9 Implementation Pipeline

1. Detect four IR blobs in the camera image.
2. Associate them with known screen coordinates.
3. Build and solve the linear system for H .
4. Use H to map new detections (u, v) into screen coordinates.
5. Quantify error via induced norm in $\mathbb{R}^2$.

10 Conclusion

The inner product space framework provides the mathematical context to describe camera geometry in terms of rays, angles, and distances, while orthogonal projection offers the geometric link between rays and the screen plane. Homography then encodes this relationship into a computationally efficient linear operator, allowing us to implement the mapping in practice.

References

- [1] Wikipedia: *Gaussian Elimination*. https://en.wikipedia.org/wiki/Gaussian_elimination
- [2] OpenCV: *Homography Tutorial*. https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html